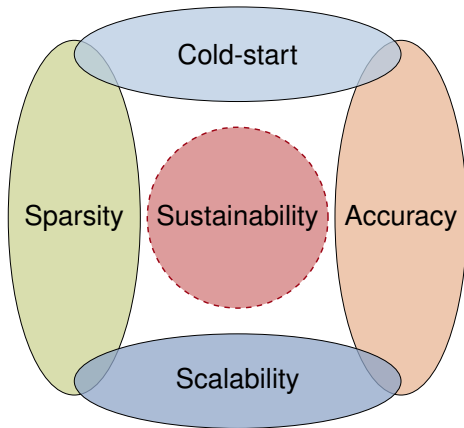


Energy-Aware Hybrid Recommender Systems Across the User Lifecycle

Saina Amiri Moghadam

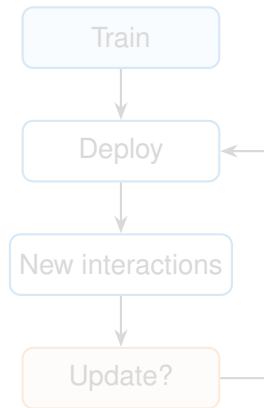
Chair of Connected Mobility
TUM School of Computation, Information and
Technology
Technical University of Munich

October 9, 2026



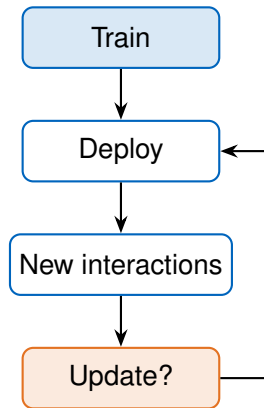
Introduction

- Recommender models are becoming more expensive without proportionate gains in quality
- A recommender is not trained once: new users arrive and preferences shift, so the model must be updated
- The recurring carbon cost of updating has not been measured

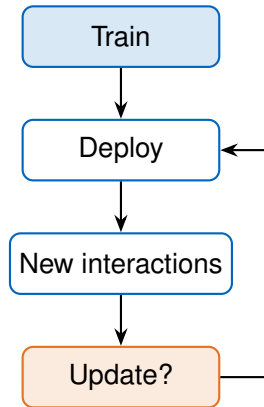


Introduction

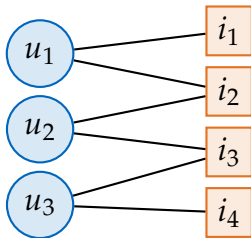
- Recommender models are becoming more expensive without proportionate gains in quality
- A recommender is not trained once: new users arrive and preferences shift, so the model must be updated
- The recurring carbon cost of updating has not been measured



- Recommender models are becoming more expensive without proportionate gains in quality
- A recommender is not trained once: new users arrive and preferences shift, so the model must be updated
- The recurring carbon cost of updating has not been measured



What does it cost to keep a recommender up to date, and can a cheaper hybrid mechanism deliver comparable quality?



User–Item Interaction Graph

Neighborhood aggregation:

$$\mathbf{e}_u^{(k)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \sqrt{|\mathcal{N}_i|}} \mathbf{e}_i^{(k-1)}$$

$$\text{Prediction: } \hat{y}_{ui} = \mathbf{e}_u^\top \mathbf{e}_i$$

CPU and RAM energy estimated
with CodeCarbon

$$\text{emissions} = C \cdot E$$

$$C = \sum_{s \in S} e_s \cdot p_s$$

$$C \text{ (Germany)} = 381 \text{ g CO}_2\text{eq/kWh}$$

CPU and RAM energy estimated
with CodeCarbon

$$\text{emissions} = C \cdot E$$

$$C = \sum_{s \in S} e_s \cdot p_s$$

$$C (\text{Germany}) = 381 \text{ g CO}_2\text{eq/kWh}$$

CPU and RAM energy estimated
with CodeCarbon

$$\text{emissions} = C \cdot E$$

$$C = \sum_{s \in S} e_s \cdot p_s$$

$$C (\text{Germany}) = 381 \text{ g CO}_2\text{eq/kWh}$$

Evaluation Metrics

Per batch: score all items,
mask items already seen,
recommend the top 10

Relevant: items the user
liked in this batch

$$\text{Precision@10} = \frac{\text{relevant items retrieved}}{10}$$

$$\text{Recall@10} = \frac{\text{relevant items retrieved}}{\text{relevant items}}$$

$$\text{NDCG@10} = \frac{\text{hits, weighted by rank}}{\text{best possible ranking}}$$

Evaluation Metrics

Per batch: score all items,
mask items already seen,
recommend the top 10

Relevant: items the user
liked in this batch

$$\text{Precision@10} = \frac{\text{relevant items retrieved}}{10}$$

$$\text{Recall@10} = \frac{\text{relevant items retrieved}}{\text{relevant items}}$$

$$\text{NDCG@10} = \frac{\text{hits, weighted by rank}}{\text{best possible ranking}}$$

Per batch: score all items,
mask items already seen,
recommend the top 10

Relevant: items the user
liked in this batch

$$\text{Precision@10} = \frac{\text{relevant items retrieved}}{10}$$

$$\text{Recall@10} = \frac{\text{relevant items retrieved}}{\text{relevant items}}$$

$$\text{NDCG@10} = \frac{\text{hits, weighted by rank}}{\text{best possible ranking}}$$

Per batch: score all items,
mask items already seen,
recommend the top 10

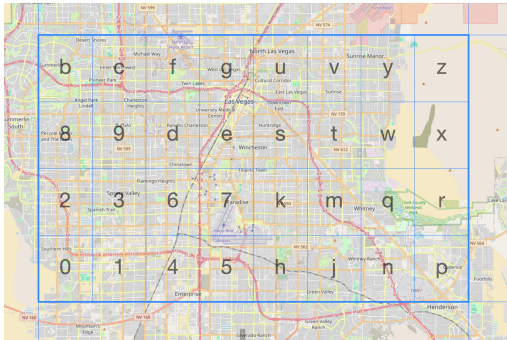
Relevant: items the user
liked in this batch

$$\text{Precision@10} = \frac{\text{relevant items retrieved}}{10}$$

$$\text{Recall@10} = \frac{\text{relevant items retrieved}}{\text{relevant items}}$$

$$\text{NDCG@10} = \frac{\text{hits, weighted by rank}}{\text{best possible ranking}}$$

Content Signals: Location and Category



- **Location:** geohash cells, nearby businesses share a prefix
- **Category:** business category
- **MovieLens:** genre and release decade instead

Content-Based Initialization

1 Profile trained users

	g_1	g_2	g_3	g_4
u_1	3	0	1	0
u_2	0	2	0	1
u_3	1	0	4	0
u_4	0	0	1	2

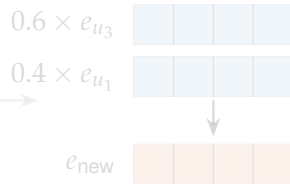
check-ins per geohash
(same for categories)

2 Score similarity

new user	1	0	2	0
u_1	0.67			
u_2	0.00			
u_3	1.00			
u_4	0.33			

min-sum overlap
 $S = 0.7 \cdot \text{geo} + 0.3 \cdot \text{cat}$

3 Initialize embedding



weights = normalized scores
of the top- k users ($k = 20$)

Content-Based Initialization

1 Profile trained users

	g_1	g_2	g_3	g_4
u_1	3	0	1	0
u_2	0	2	0	1
u_3	1	0	4	0
u_4	0	0	1	2

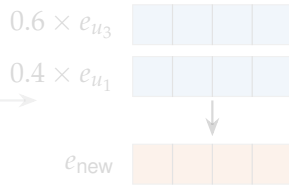
check-ins per geohash
(same for categories)

2 Score similarity

new user	1	0	2	0
u_1				0.67
u_2				0.00
u_3				1.00
u_4				0.33

min-sum overlap
 $S = 0.7 \cdot \text{geo} + 0.3 \cdot \text{cat}$

3 Initialize embedding



weights = normalized scores
of the top- k users ($k = 20$)

Content-Based Initialization

1 Profile trained users

	g_1	g_2	g_3	g_4
u_1	3	0	1	0
u_2	0	2	0	1
u_3	1	0	4	0
u_4	0	0	1	2

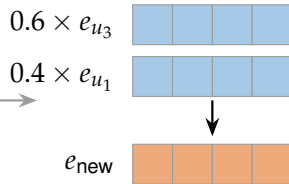
check-ins per geohash
(same for categories)

2 Score similarity

new user	1	0	2	0
u_1	0.67			
u_2	0.00			
u_3	1.00			
u_4	0.33			

min-sum overlap
 $S = 0.7 \cdot \text{geo} + 0.3 \cdot \text{cat}$

3 Initialize embedding



weights = normalized scores
of the top- k users ($k = 20$)

Incremental Update:

Keeps trained users up to
date

Content-Based Initialization:

Improves performance for
untrained users



Combine both and compare quality against carbon
emissions

Incremental Update:

Keeps trained users up to
date

Content-Based Initialization:

Improves performance for
untrained users



Combine both and compare quality against carbon
emissions

Methodology

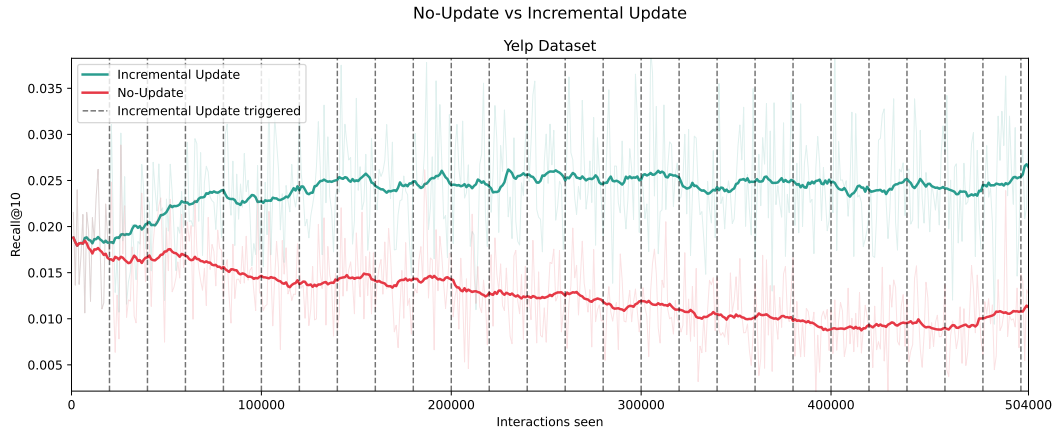
- Split Yelp and MovieLens along the global timeline: 80% training, 20% streamed in batches
- Compare no-update, incremental, content-based initialization, their combination, and full retraining
- Measure Recall@10 per batch and emissions per phase with CodeCarbon

Methodology

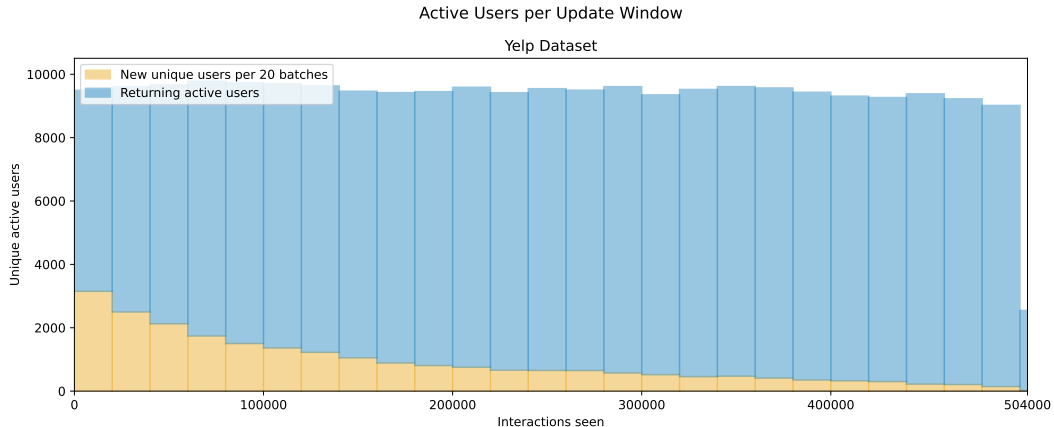
- Split Yelp and MovieLens along the global timeline: 80% training, 20% streamed in batches
- Compare no-update, incremental, content-based initialization, their combination, and full retraining
- Measure Recall@10 per batch and emissions per phase with CodeCarbon

- Split Yelp and MovieLens along the global timeline: 80% training, 20% streamed in batches
- Compare no-update, incremental, content-based initialization, their combination, and full retraining
- Measure Recall@10 per batch and emissions per phase with CodeCarbon

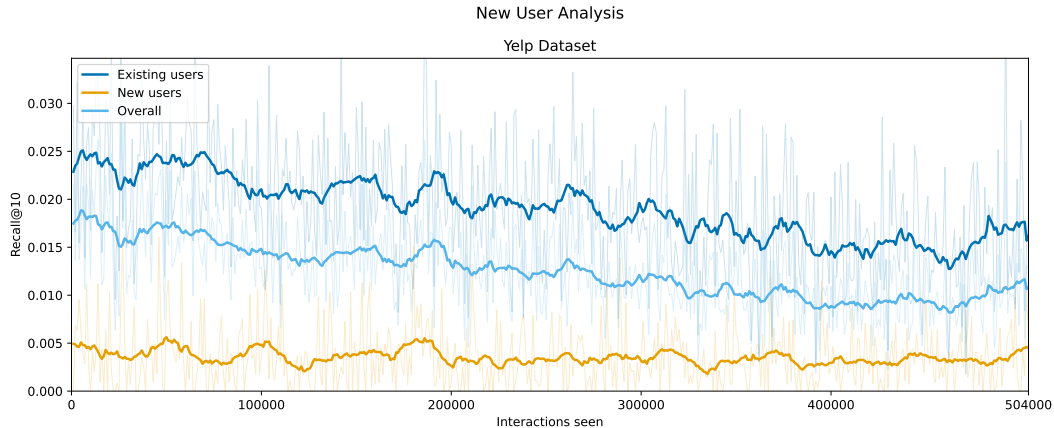
Yelp: No-Update vs. Incremental Update



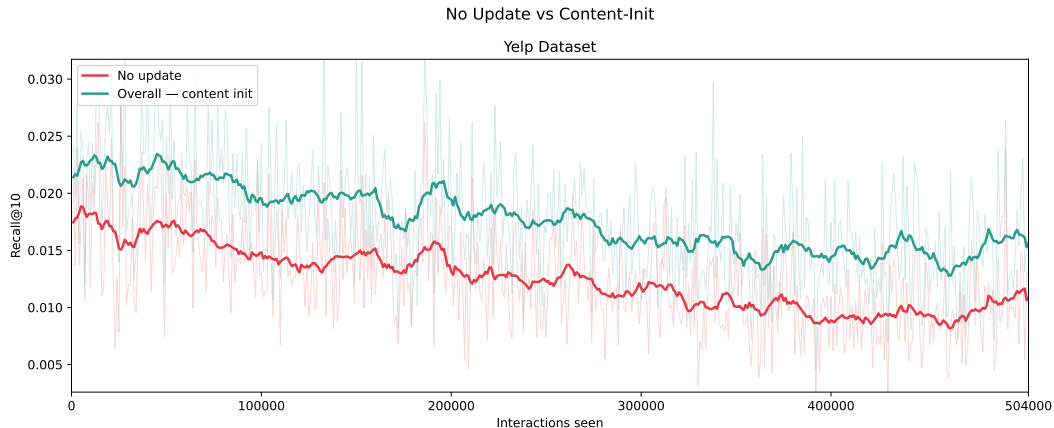
Yelp: Active Users per Update Window



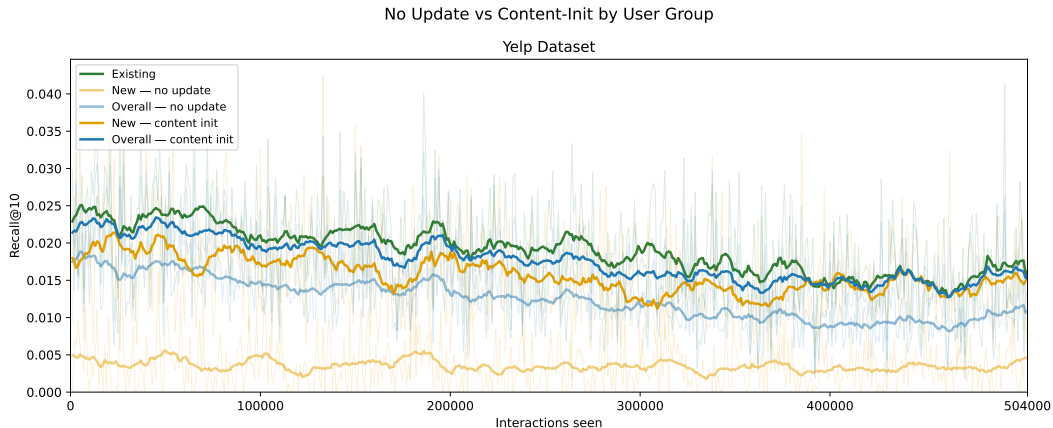
Yelp: Existing vs. New Users



Yelp: No-Update vs. Content-Based Initialization

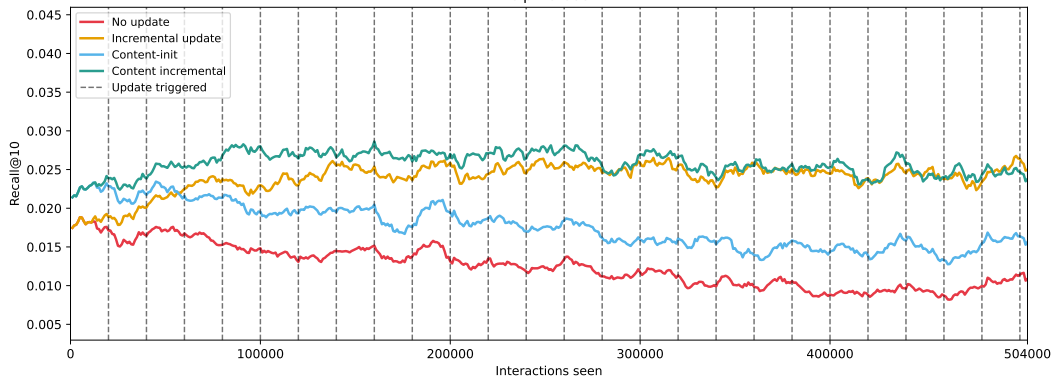


Yelp: No-Update vs. Content-Based Initialization

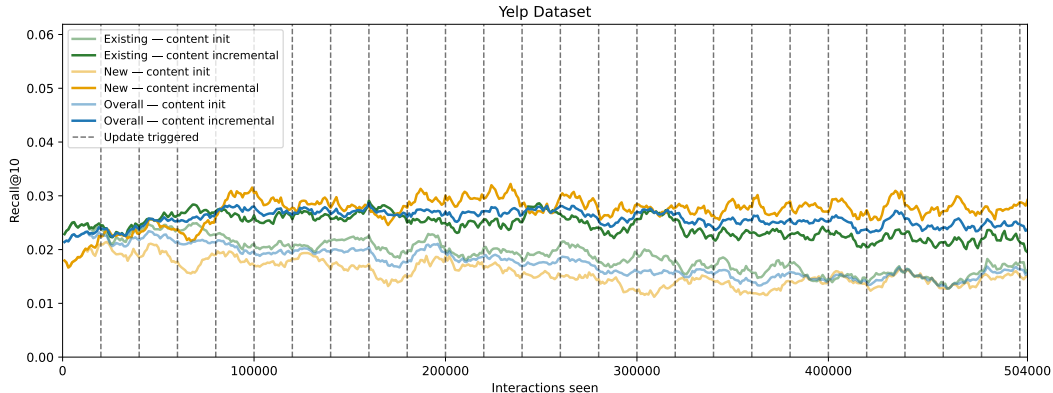


Strategy Comparison

Yelp Dataset



Content-Init vs Content Incremental by User Group



Comparison of Strategies

	No-Update	Incremental	Content-Init	Content + Incremental
Recall@10	0.0125	+91%	+41%	+106%
Emissions (% of training)	5.20%	6.82%	6.14%	7.55%

	No-Update	Incremental	Content-Init	Content + Incremental
Recall@10	0.0125	+91%	+41%	+106%
Emissions (% of training)	5.20%	6.82%	6.14%	7.55%

	No-Update	Incremental	Content-Init	Content + Incremental
Recall@10	0.0125	+91%	+41%	+106%
Emissions (% of training)	5.20%	6.82%	6.14%	7.55%

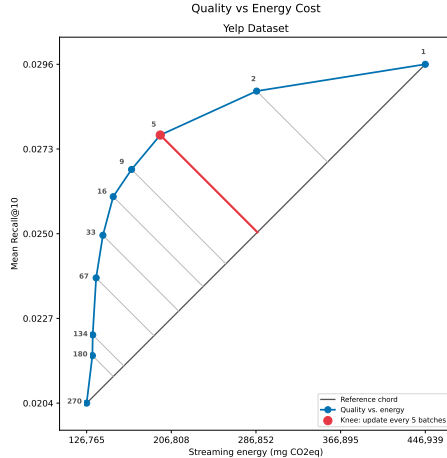
Comparison of Strategies

	No-Update	Incremental	Content-Init	Content + Incremental
Recall@10	0.0125	+91%	+41%	+106%
Emissions (% of training)	5.20%	6.82%	6.14%	7.55%

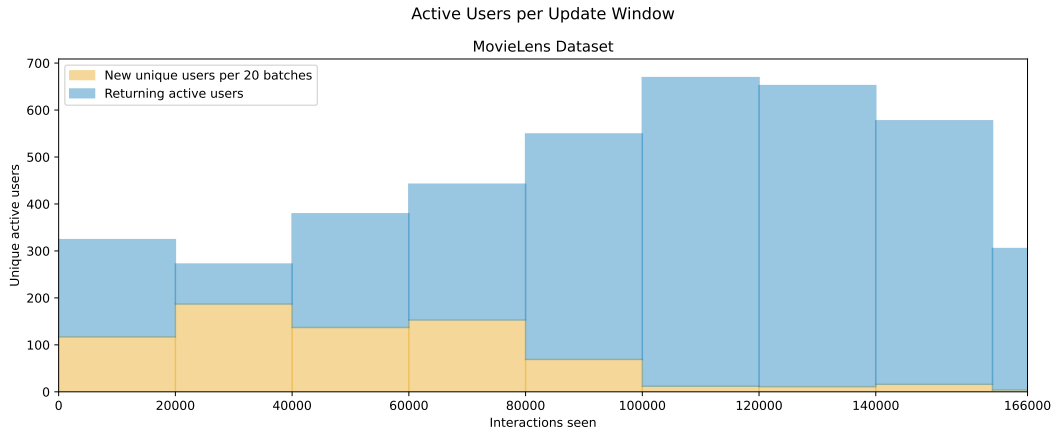
Comparison of Strategies

	No-Update	Incremental	Content-Init	Content + Incremental
Recall@10	0.0125	+91%	+41%	+106%
Emissions (% of training)	5.20%	6.82%	6.14%	7.55%

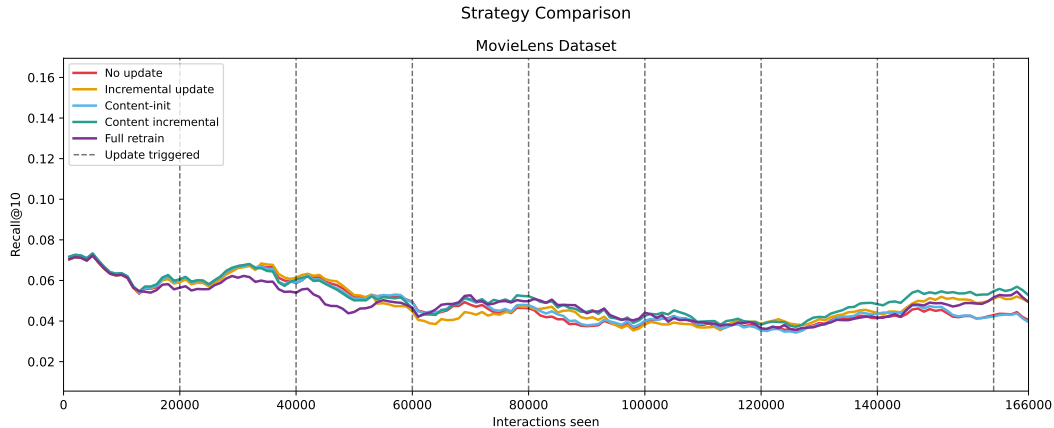
Yelp: Update Frequency vs. Emissions



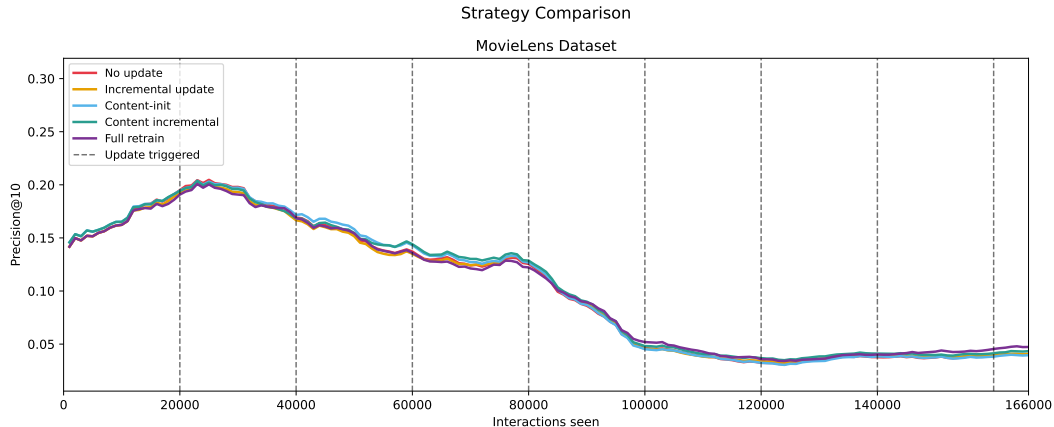
MovieLens: Active Users per Update Window



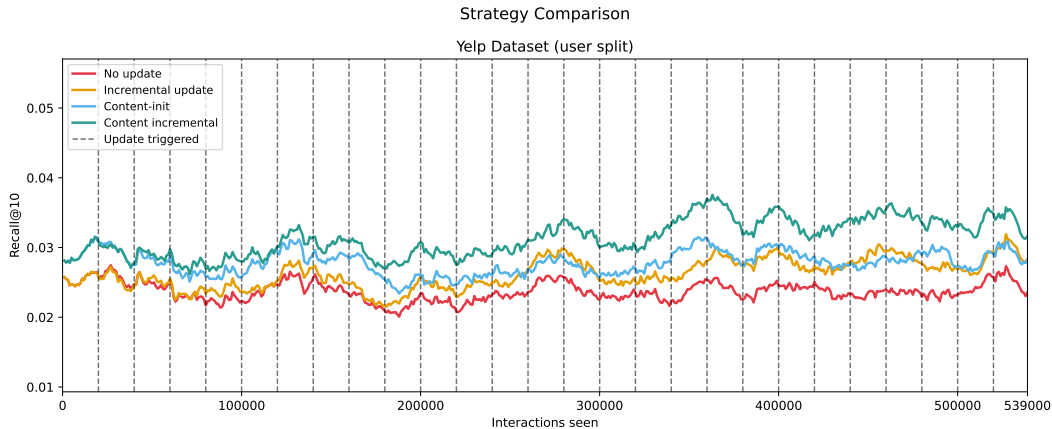
MovieLens: All Strategies



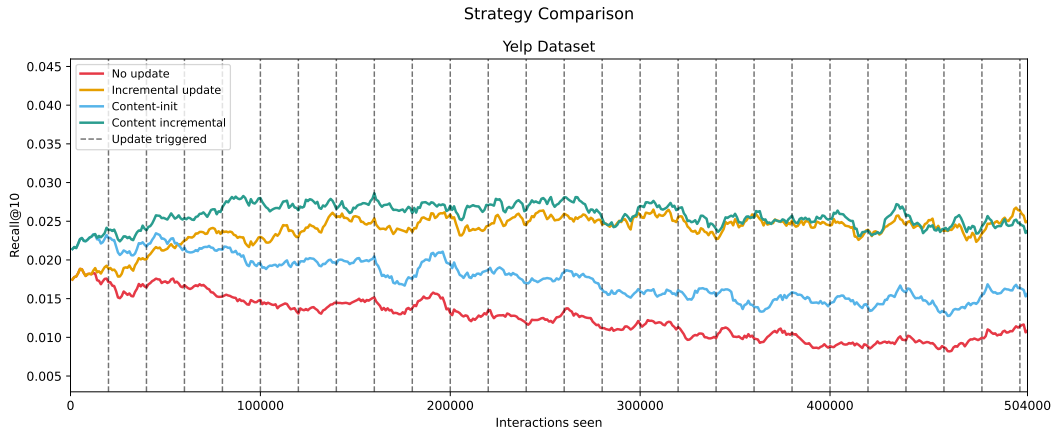
MovieLens: Precision@10, All Strategies



Pitfall: Per-User Split



Pitfall: Per-User Split



Cheap updates recover most lost quality, but only where there is decay to correct



Diagnose the source of decay before choosing an update strategy

Cheap updates recover most lost quality, but only where there is decay to correct



Diagnose the source of decay before choosing an update strategy

Next Steps

- Causal incremental graph convolution (CI-LightGCN), measured in energy rather than time
- Windowed graph construction to bound the update cost
- Streams with controlled, documented drift
- More datasets with less strict interaction filtering

Next Steps

- Causal incremental graph convolution (CI-LightGCN), measured in energy rather than time
- Windowed graph construction to bound the update cost
- Streams with controlled, documented drift
- More datasets with less strict interaction filtering

Next Steps

- Causal incremental graph convolution (CI-LightGCN), measured in energy rather than time
- Windowed graph construction to bound the update cost
- Streams with controlled, documented drift
- More datasets with less strict interaction filtering

Next Steps

- Causal incremental graph convolution (CI-LightGCN), measured in energy rather than time
- Windowed graph construction to bound the update cost
- Streams with controlled, documented drift
- More datasets with less strict interaction filtering

Thank you!

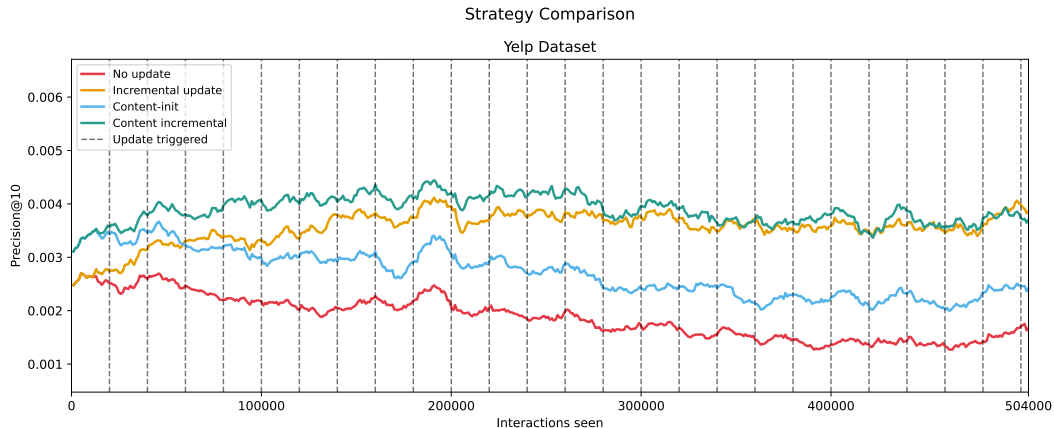
Chair of Connected Mobility

Examiner: Prof. Dr.-Ing. Jörg Ott

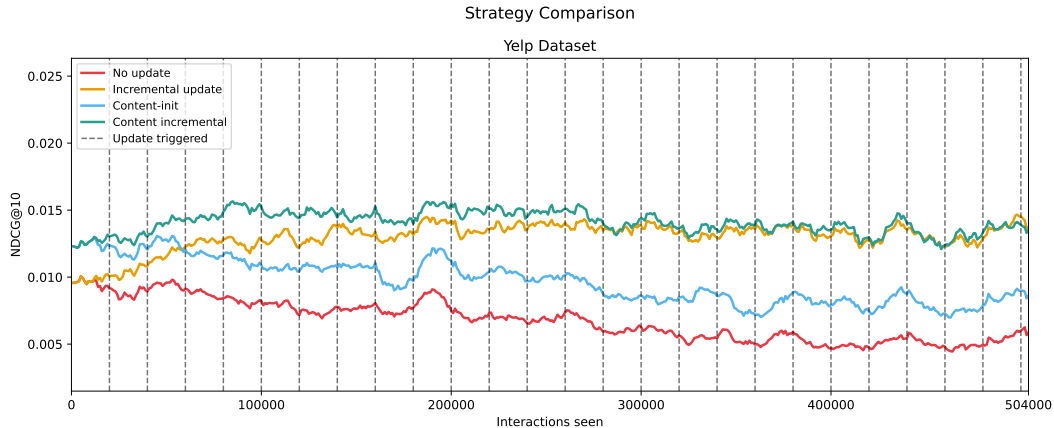
Advisor: Prof. Wolfgang Wörndl

Appendix

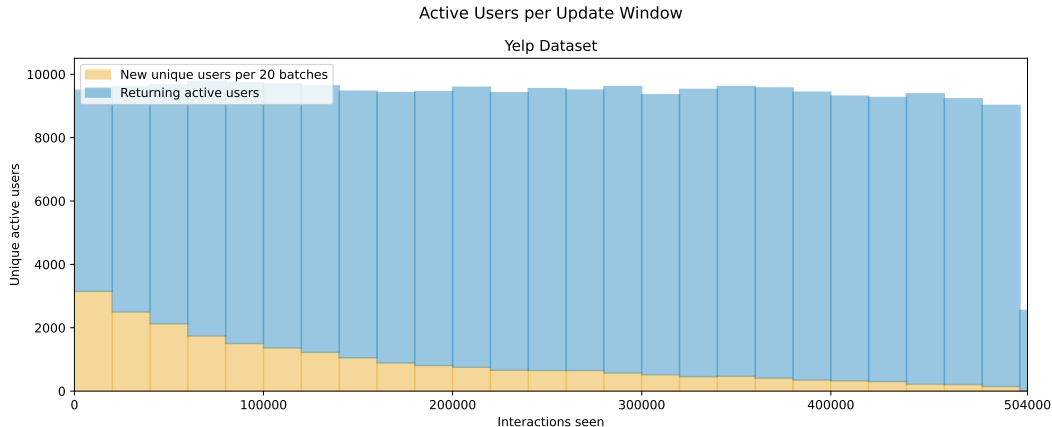
Yelp: Precision@10, All Strategies



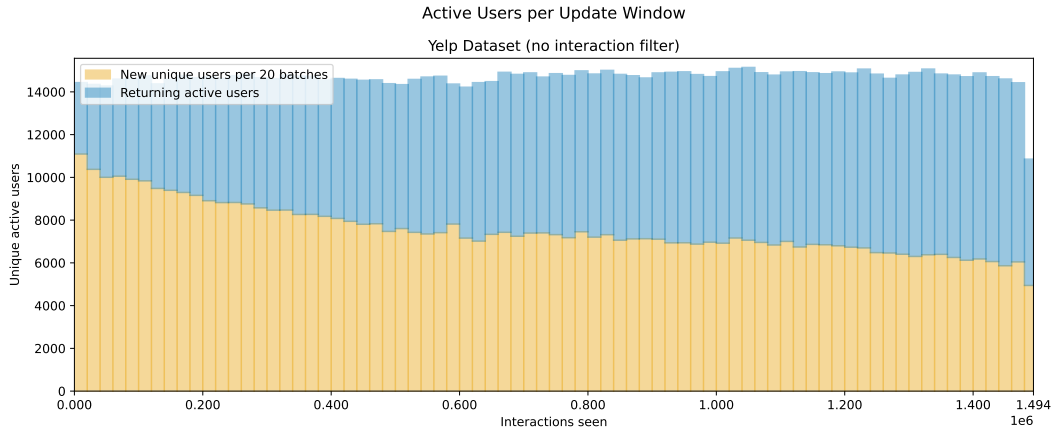
Yelp: NDCG@10, All Strategies



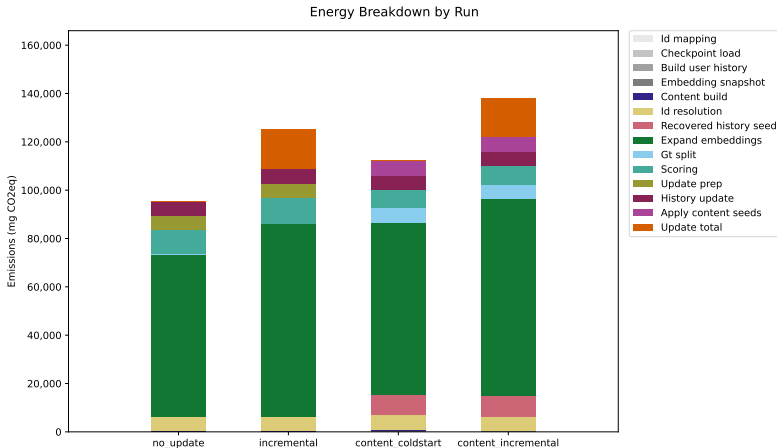
Yelp: New Unique Users per Update Window



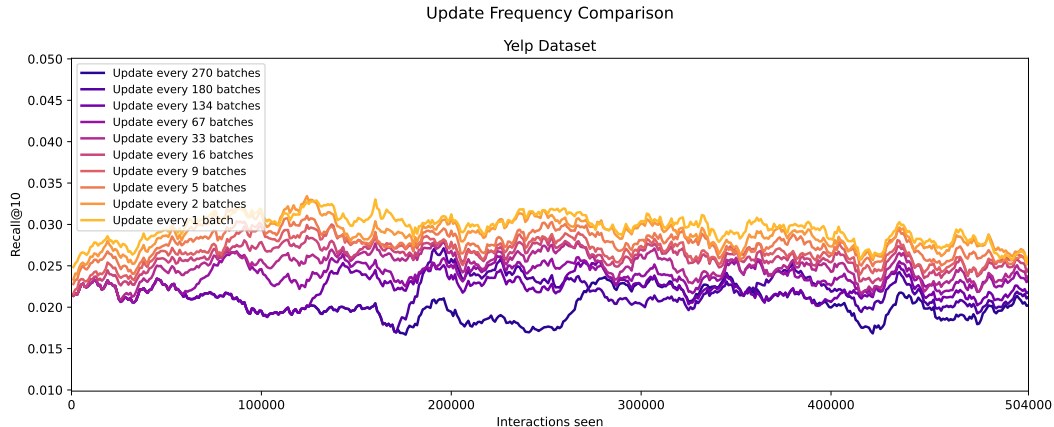
Yelp: New Unique Users per Update Window



Yelp: Emissions by Phase



Yelp: Update Frequency Variants



Carbon Cost per Strategy

	No-Update	Incremental	Content-Init	Content + Incremental
Yelp streaming	95,306	125,053	112,422	138,321
Yelp training		1,832,436		
MovieLens streaming	17,286	18,654	20,045	22,618
MovieLens training		188,839		

Carbon emissions in mg CO₂eq. Full retrain on MovieLens: 1,387,568 streaming, 792× incremental per update.

References I

- [1] G. Spillo, A. De Filippo, C. Musto, M. Milano, and G. Semeraro, in Proceedings of the 17th acm conference on recommender systems, RecSys '23 (2023), pp. 856–862.
- [2] T. Vente, L. Wegmeth, A. Said, and J. Beel, in Proceedings of the 18th acm conference on recommender systems, RecSys '24 (2024), pp. 580–590.
- [3] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, in Proceedings of the 43rd international acm sigir conference on research and development in information retrieval, SIGIR '20 (2020), pp. 639–648.
- [4] CodeCarbon, *Methodology*, (2024)
<https://docs.codecarbon.io/latest/explanation/methodology/>.
- [5] G. Niemeyer, *Geohash.org is public!*, (2008)
<https://web.archive.org/web/20080305223755/http://blog.labix.org/#post-84>.

References II

- [6] Geohash Explorer, *Geohash 9qqj*, <https://geohash.softeng.co/9qqj>.
- [7] E. Çano and M. Morisio, Intelligent Data Analysis **21**, 1487 (2017).
- [8] V. Satopaä, J. Albrecht, D. Irwin, and B. Raghavan, in 2011 31st international conference on distributed computing systems workshops (2011), pp. 166–171.
- [9] F. M. Harper and J. A. Konstan, ACM Transactions on Interactive Intelligent Systems **5**, 19 (2015).
- [10] Y. Ji, A. Sun, J. Zhang, and C. Li, ACM Transactions on Information Systems **41**, 75, 1 (2023).
- [11] S. Ding, F. Feng, X. He, Y. Liao, J. Shi, and Y. Zhang, IEEE Transactions on Neural Networks and Learning Systems **35**, 4718 (2024).

References III

- [12] A. Ghiye, B. Barreau, L. Carlier, and M. Vazirgiannis, in Proceedings of the 5th acm international conference on ai in finance, ICAIF '24 (2024), pp. 231–238.
- [13] V. M. A. Souza, D. M. dos Reis, A. G. Maletzke, and G. E. A. P. A. Batista, Data Mining and Knowledge Discovery **34**, 1805 (2020).